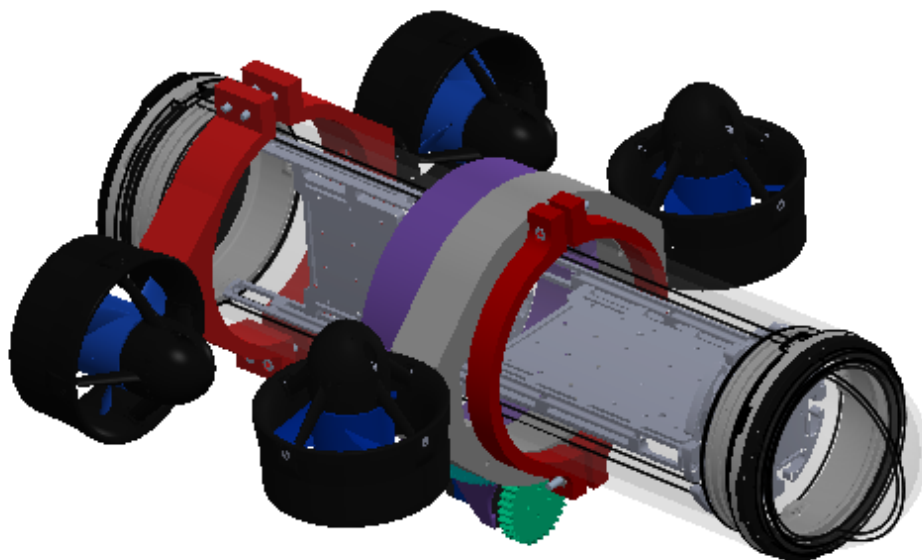


## Rapport d'avancement

---

# PROJET 3A - DÉVELOPPEMENT DU ROBOT PLASMAR

---



BISBAU Txomin, MANU Léa, TOURI Maha

Professeur référent : DEGORRE Loïck

21 juillet 2026

# Résumé

Ce rapport présente l'état d'avancement du projet PlaSMAR, dont l'objectif est l'évolution d'un robot sous-marin initialement téléopéré vers un système autonome et reconfigurable. Les travaux réalisés portent sur la définition du cahier des charges, la refonte de l'architecture matérielle et logicielle, ainsi que sur l'intégration progressive des différents sous-systèmes. Une part importante du projet concerne l'utilisation de BlueOS et de ROS 2 afin de permettre un contrôle modulaire des actionneurs, une supervision flexible et la préparation à des lois de commande plus avancées. En parallèle, un environnement de simulation a été développé pour modéliser le comportement dynamique du robot et valider les stratégies de commande avant leur déploiement sur le système réel. Les résultats obtenus à ce stade montrent la cohérence de l'architecture retenue et ouvrent la voie à des essais expérimentaux en conditions réelles.

**Mots-clés :** Robot sous-marin ; Transmission magnétique ; ROS 2 ; BlueOS ; Simulation.

# Abstract

This report presents the current progress of the PlaSMAR project, which aims to evolve an underwater robot from a manually operated platform into an autonomous and reconfigurable system. The work focuses on requirement definition, hardware and software architecture redesign, and the progressive integration of the different subsystems. A significant part of the project relies on the use of BlueOS and ROS 2 to enable modular actuator control, flexible supervision, and the future implementation of advanced control laws. In parallel, a simulation environment has been developed to model the robot's dynamic behavior and to validate control strategies prior to deployment on the real system. The results obtained so far confirm the relevance of the chosen architecture and provide a solid basis for upcoming experimental trials.

**Keywords :** Underwater robot ; Magnetic transmission ; ROS 2 ; BlueOS ; Simulation.

## Remerciements

Par ces quelques lignes, l'équipe tient à exprimer sa sincère gratitude envers les personnes qui ont grandement contribué à la réalisation de ce projet. Grâce à leur soutien, nous avons pu allier rigueur scientifique et gestion d'un projet complexe avec succès.

Nous souhaitons remercier particulièrement :

En premier lieu, Monsieur Loïck DEGORRE, notre tuteur de projet, pour sa confiance, sa disponibilité, son aide et pour nous avoir permis de travailler et de développer un nouveau robot PlaSMAR.

Nous tenons également à exprimer notre reconnaissance envers Monsieur Cédric ANTHIERENS, pour son implication en tant que responsable du parcours SYSMER, qui a été d'une grande aide tout au long de notre démarche.

Sans leur soutien, nos progrès n'auraient pas été aussi marqués, et nous leur en sommes profondément reconnaissants.

# Table des matières

|                                                                               |           |
|-------------------------------------------------------------------------------|-----------|
| <b>Glossaire</b>                                                              | <b>5</b>  |
| <b>1 Introduction</b>                                                         | <b>7</b>  |
| <b>2 Cahier des charges</b>                                                   | <b>7</b>  |
| 2.1 Exigences électroniques et logicielles . . . . .                          | 8         |
| 2.2 Exigences mécaniques . . . . .                                            | 8         |
| 2.3 Exigences de modélisation et validation . . . . .                         | 8         |
| 2.4 Exigences générales . . . . .                                             | 9         |
| <b>3 Architecture matérielle</b>                                              | <b>10</b> |
| 3.1 État de l’art et Limitations du Prototype Précédent . . . . .             | 10        |
| 3.2 Nouvelle Architecture proposée . . . . .                                  | 10        |
| 3.2.1 Unité de Calcul et de Contrôle . . . . .                                | 11        |
| 3.2.2 Énergie . . . . .                                                       | 11        |
| 3.2.3 Actionnement et Transmission . . . . .                                  | 12        |
| <b>4 Conception et Amélioration Mécanique</b>                                 | <b>12</b> |
| 4.1 Modèle à transmission mécanique . . . . .                                 | 12        |
| 4.1.1 Système final . . . . .                                                 | 13        |
| 4.2 Modèle à transmission magnétique . . . . .                                | 13        |
| 4.2.1 Estimation du couple de transmission magnétique minimal nécessaire      | 14        |
| 4.2.2 Calculs du couple de transmission magnétique . . . . .                  | 14        |
| 4.2.3 Calculs du poids des aimants . . . . .                                  | 15        |
| 4.2.4 Système final . . . . .                                                 | 15        |
| 4.3 État actuel . . . . .                                                     | 16        |
| <b>5 Modélisation et simulation du robot</b>                                  | <b>16</b> |
| 5.1 Première version : Validation architecturale (Proof of Concept) . . . . . | 16        |
| 5.2 Deuxième version : Intégration du Modèle Physique Complet . . . . .       | 17        |
| 5.2.1 Masse et Inertie $M$ . . . . .                                          | 17        |
| 5.2.2 Effet de Coriolis $C$ . . . . .                                         | 18        |
| 5.2.3 Amortissement Hydrodynamique $D$ . . . . .                              | 18        |
| 5.2.4 Gravité et Stabilité $g$ . . . . .                                      | 18        |
| 5.2.5 Loi de Commande . . . . .                                               | 18        |
| 5.2.6 Implémentation logicielle de la simulation . . . . .                    | 19        |
| 5.2.7 Limitations . . . . .                                                   | 19        |
| 5.3 Troisième version : Banc de Test et Monitoring . . . . .                  | 20        |
| 5.3.1 Protocoles de Test . . . . .                                            | 20        |
| 5.3.2 Monitoring . . . . .                                                    | 20        |
| 5.3.3 Caractérisation et modélisation des actionneurs . . . . .               | 20        |
| 5.3.4 Modélisation de l’anneau et allocation . . . . .                        | 21        |
| 5.3.5 Résultats . . . . .                                                     | 21        |
| 5.4 Version Finale : Suivi de trajectoire et mode autonome . . . . .          | 22        |
| 5.4.1 Séquencement de la mission . . . . .                                    | 22        |
| 5.4.2 Amélioration de la loi de commande . . . . .                            | 22        |
| 5.4.3 Validation de l’architecture (RQT Graph) . . . . .                      | 23        |

|          |                                                                   |           |
|----------|-------------------------------------------------------------------|-----------|
| 5.4.4    | Résultats Simulation . . . . .                                    | 23        |
| <b>6</b> | <b>Réalisation et Intégration</b>                                 | <b>25</b> |
| 6.1      | Intégration matérielle et câblage . . . . .                       | 25        |
| 6.1.1    | Rôle des ESC et distribution de puissance . . . . .               | 26        |
| 6.1.2    | Choix des branchements sur le Flight Navigator . . . . .          | 26        |
| 6.2      | Configuration de BlueOS et du contrôleur de vol . . . . .         | 26        |
| 6.2.1    | Connexion à l'interface BlueOS . . . . .                          | 27        |
| 6.2.2    | Ajout de ROS2 et compatibilité système . . . . .                  | 27        |
| 6.2.3    | Configuration des sorties PWM et mode RC Override . . . . .       | 27        |
| 6.3      | Déploiement et architecture logicielle ROS2 . . . . .             | 27        |
| 6.4      | Logique de commande et allocation des actionneurs . . . . .       | 29        |
| 6.4.1    | Tableau de vérité des actions élémentaires . . . . .              | 29        |
| 6.4.2    | Implémentation logicielle et allocation des actionneurs . . . . . | 29        |
| 6.5      | Commande par allocation matricielle des propulseurs . . . . .     | 30        |
| <b>7</b> | <b>Conclusion et perspectives</b>                                 | <b>31</b> |
| 7.1      | Synthèse et validation du système . . . . .                       | 31        |
| 7.2      | Pistes d'améliorations . . . . .                                  | 31        |
| 7.2.1    | Transmission magnétique . . . . .                                 | 31        |
| 7.2.2    | Essais expérimentaux en bassin . . . . .                          | 31        |
| 7.2.3    | Autonomie de navigation et fusion de capteurs . . . . .           | 31        |
| <b>A</b> | <b>Annexes</b>                                                    | <b>34</b> |
| A.1      | Workspace ROS2 et codes sources . . . . .                         | 34        |
| A.2      | Résultats de simulation complémentaires . . . . .                 | 34        |
| A.2.1    | Stabilité passive . . . . .                                       | 34        |
| A.2.2    | Visualisations 3D des Mouvements Élémentaires . . . . .           | 35        |
| A.2.3    | Séquence de mouvement . . . . .                                   | 35        |
| A.3      | Commandes aimants . . . . .                                       | 36        |

## Glossaire

- **Actionneur (Thruster)** : Organe mécanique (ici, les moteurs T200 et l’anneau rotatif) convertissant une commande électrique en une force physique pour déplacer le robot.
- **AUV (Autonomous Underwater Vehicle)** : Robot sous-marin autonome, capable de réaliser une mission sans intervention humaine ni câble de liaison, contrairement aux ROV téléopérés.
- **BlueOS** : Système d’exploitation basé sur Linux développé par BlueRobotics, tournant sur la Raspberry Pi. Il gère l’interface web, les drivers matériels et la communication MAVLink.
- **Boucle Fermée (Closed Loop)** : Principe de contrôle où la commande envoyée aux moteurs est corrigée en permanence en fonction de la mesure réelle des capteurs pour minimiser l’erreur.
- **Docker** : Plateforme logicielle de conteneurisation. Elle permet d’isoler l’environnement de simulation (ROS 2, Gazebo) pour garantir qu’il fonctionne à l’identique sur n’importe quel ordinateur.
- **ESC (Electronic Speed Controller)** : Variateur électronique de vitesse. Carte électronique de puissance qui contrôle la rotation des moteurs brushless (triphasés) en fonction d’un signal de commande (PWM).
- **Foxglove** : Outil de visualisation de données pour la robotique. Utilisé ici pour afficher en temps réel l’état du robot (trajectoires, courbes) via une connexion WebSocket.
- **IMU (Inertial Measurement Unit)** : Centrale inertielle embarquée sur le contrôleur de vol. Elle combine accéléromètres, gyroscopes et magnétomètres pour estimer l’orientation du robot (Roulis, Tangage, Lacet).
- **Jacobienne ( $J$ )** : Matrice de transformation mathématique permettant de passer des vitesses exprimées dans le repère du robot (Body frame) aux vitesses exprimées dans le repère terrestre (World frame).
- **MAVLink** : Protocole de communication léger et standardisé, utilisé pour échanger des données (télémétrie, commandes, paramètres) entre le contrôleur de vol (Navigator) et l’ordinateur de bord.
- **Nœud (Node)** : Unité fondamentale d’exécution dans ROS 2. Chaque nœud est un programme indépendant responsable d’une tâche spécifique (ex : lecture capteur, calcul de trajectoire, pilotage moteur).
- **Odométrie** : Estimation de la position et de l’orientation du robot au cours du temps, calculée ici par intégration mathématique des vitesses simulées.
- **PID (Proportionnel Intégral Dérivé)** : Algorithme de régulation utilisé pour asservir la position du robot. Il corrige l’erreur via trois termes : proportionnel (P), intégral (I) et dérivé (D).
- **PWM (Pulse Width Modulation)** : Modulation de Largeur d’Impulsion. Technique utilisée pour coder une information analogique (la vitesse du moteur) dans un signal numérique en faisant varier la durée de l’état haut du signal.
- **ROS 2 (Robot Operating System)** : Middleware open-source utilisé pour le développement d’applications robotiques. Il gère la communication entre les différents composants via un système de messages (Topics).
- **SITL (Software-In-The-Loop)** : Méthode de simulation où le code réel du contrôleur est exécuté contre un modèle simulé du robot, sans le matériel phy-

sique, permettant de valider la logique avant les essais réels.

- **Topic** : Canal de communication nommé sur lequel les nœuds ROS échangent des messages de manière asynchrone (architecture Publish/Subscribe).
- **Réseau Halbach** : agencement spécifique d'aimants permanents qui amplifie et oriente le champ magnétique d'un côté d'une structure tout en l'annulant presque totalement de l'autre.

# 1 Introduction

L'exploration des milieux sous-marins nécessite des robots capables d'être maniables, endurants et autonomes. Dans ce contexte, les AUV (Autonomous Underwater Vehicles) sous-actionnés, possédant moins d'actionneurs que de degrés de liberté, représentent un défi technologique majeur, car ils requièrent un contrôle avancé pour compenser ce manque d'actionnement.

Le robot PlaSMAR (Plateforme Sous-Marine Autonome Reconfigurable) s'inscrit dans cette dynamique. Il se distingue par un anneau rotatif supportant deux propulseurs, ce qui lui offre un degré de liberté supplémentaire. Cette architecture permet notamment le déplacement latéral du robot tout en garantissant sa stabilité.

Lors d'une étude précédente, un premier prototype a été réalisé, démontrant la faisabilité mécanique du concept. Cependant, ce modèle initial présentait plusieurs limites : l'électronique était basée sur une télécommande RC, ce qui limitait l'autonomie, la flottabilité était mal maîtrisée et aucune boucle de contrôle fermée n'était implémentée pour la navigation autonome.

L'objectif de notre projet est de reproduire et d'améliorer ce prototype pour faire du robot PlaSMAR un système autonome et fiable. Ce rapport détaille notre démarche d'ingénierie, de l'analyse des besoins jusqu'aux résultats de simulation et aux premiers tests d'intégration.

## 2 Cahier des charges

Le passage d'un robot téléopéré à un AUV autonome nécessite une redéfinition des exigences du système.

Afin de définir précisément le besoin auquel répond le robot PlaSMAR, nous avons réalisé une analyse fonctionnelle. Le diagramme "bête à corne" ci-dessous 1 synthétise la mission principale du système.

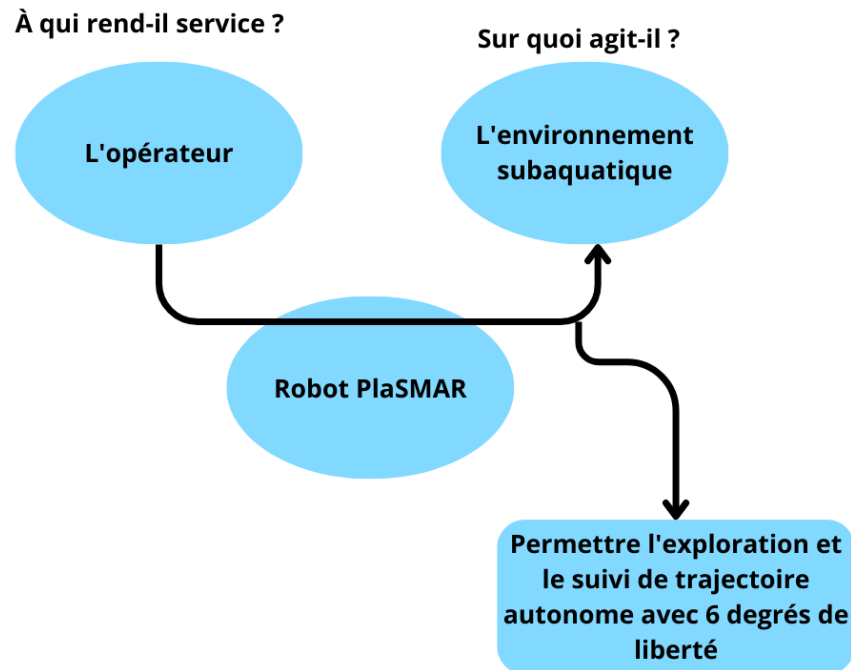


FIGURE 1 – Diagramme bête à corne

Trois problématiques majeures ont ainsi été identifiées afin de structurer cette évolution :

## 2.1 Exigences électroniques et logicielles

Le système doit intégrer une unité de calcul embarquée suffisamment puissante pour exécuter des algorithmes de navigation complexes et déployer des nœuds ROS2. Il doit aussi disposer d'une IMU pour l'asservissement ainsi qu'une liaison sans fil. Ces mises à jour sont essentielles pour le fonctionnement autonome du robot.

## 2.2 Exigences mécaniques

Le système de rotation de l'anneau par engrenage imprimé en 3D présentait des défauts d'alignement, des frottements et une usure prématurée.

- **Transmission magnétique :** Le système de rotation de l'anneau doit être fiabilisé pour éliminer les frottements et les défauts d'alignement constatés. Une solution de transmission sans contact (magnétique) devra être étudiée pour garantir l'étanchéité et réduire l'usure mécanique.
- **Équilibrage et flottabilité :** L'agencement interne en CAO doit être repensé pour intégrer les nouveaux composants tout en corrigeant la flottabilité (trop importante sur la version précédente) et en équilibrant le centre de gravité et de poussée.

## 2.3 Exigences de modélisation et validation

Avant toute expérimentation en bassin, le comportement dynamique du robot doit être validé par simulation. Le système doit disposer d'un modèle dynamique complet intégrant masse, inertie et amortissement hydrodynamique, permettre la simulation de l'allocation des propulseurs, valider les commandes sous ROS2 et reproduire fidèlement les mouvements du robot en environnement virtuel.

## 2.4 Exigences générales

De façon générale, le robot doit également respecter certaines contraintes telles que l'étanchéité des composants critiques, l'autonomie énergétique suffisante pour les essais, et une architecture modulaire qui facilite l'intégration et la maintenance.

Afin de répondre efficacement à ces problématiques, une analyse fonctionnelle a été réalisée 2. Elle permet d'identifier les fonctions principales et les fonctions contraintes auxquelles le système doit se conformer pour garantir un fonctionnement fiable et adapté à son environnement.

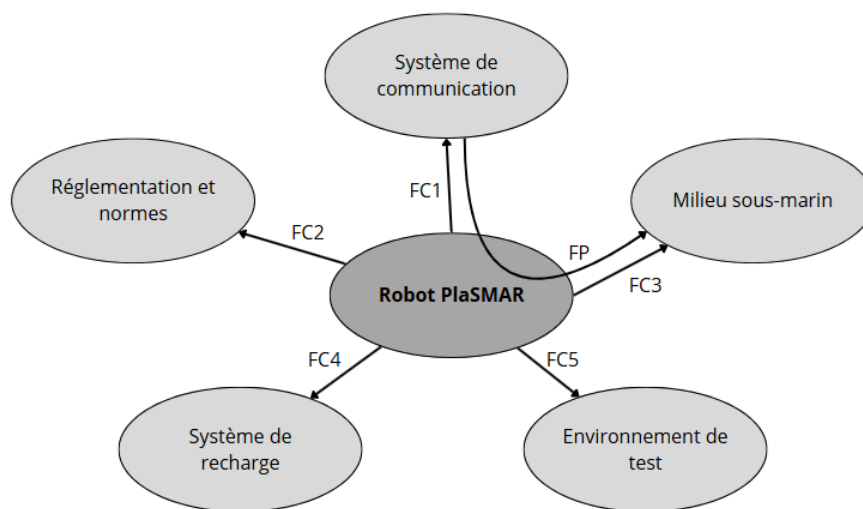


FIGURE 2 – Diagramme pieuvre

### Fonctions principales (FP)

- **FP** : Se déplacer de manière autonome et stable en milieu sous-marin (translations et rotations)

### Fonctions contraintes (FC)

- **FC1** : Reconfigurer dynamiquement l'orientation des propulseurs via l'anneau rotatif
- **FC2** : Assurer l'étanchéité globale des composants critiques (électronique, batterie) à la pression de l'eau
- **FC3** : Garantir un équilibrage des masses et une flottabilité neutre ou légèrement positive
- **FC4** : Fournir l'autonomie énergétique nécessaire pour les essais en bassin
- **FC5** : Maintenir une architecture modulaire pour faciliter le montage, le démontage et la maintenance.

| Fonction   | Critère                                    | Exigence                                                                                                          |
|------------|--------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>FP</b>  | Se déplacer de manière autonome            | Contrôle sur 4 degrés de liberté (Surge, Sway/Heave, Yaw) via l'allocation de 4 moteurs T200                      |
|            | Stabilité du système                       | Boucle d'asservissement active pour contrer le déséquilibre naturel (ex : stabilisation Pitch/Roll)               |
| <b>FC1</b> | Reconfigurer l'orientation des propulseurs | Rotation de l'anneau de $-180^\circ$ à $+180^\circ$ de manière fluide                                             |
| <b>FC2</b> | Étanchéité                                 | IP68, utilisation de tubes et connecteurs étanches Blue Robotics                                                  |
| <b>FC3</b> | Équilibrage des masses et flottabilité     | IP68 / Flottabilité neutre ou légèrement positive, ajustement basé sur un volume de $0.0185 \text{ m}^3$ et 18 kg |
| <b>FC4</b> | Autonomie énergétique                      | Batterie LiPo 3S (11.1V), 5000mAh pour une autonomie de 15-20 minutes                                             |
| <b>FC5</b> | Architecture modulaire                     | Faciliter le montage, démontage et maintenance                                                                    |

### 3 Architecture matérielle

#### 3.1 État de l'art et Limitations du Prototype Précédent

Le prototype précédent du robot PlaSMAR, détaillé dans le rapport de conception 2025 [1], reposait sur une architecture orientée vers le pilotage manuel et utilisait les composants suivants :

- **Contrôle Bas niveau** : Une carte Arduino Uno ;
- **Communication** : Une télécommande Taranis Q X7 possédant une baie Nano JR qui puisse accueillir un système radio longue portée TBS Crossfire (868 MHz) ;
- **Propulsion** : 4 ESC Blue Robotics[2] et propulseurs T200 [3] ;
- **Moteurs** : Un servomoteur ROVMaker étanche High Torque 40 kg.cm monté sur la coque externe du PlaSMAR ;
- **Énergie** : Une batterie Lipo 3s (11.1 V) et un BEC 5V.

Bien qu'idéale pour le pilotage manuel, cette conception présentait certaines limitations majeures pour l'objectif d'autonomie :

- **Faible puissance de calcul** : La carte Arduino Uno ne peut pas exécuter des algorithmes complexes ni gérer les nœuds ROS 2 [4] ;
- **Absence de capteurs de navigation** : Le système ne possédait pas de centrale inertielle (IMU) précise pour l'asservissement en orientation ;
- **Connectivité** : L'absence d'une liaison WI-FI / Bluetooth ne permettait pas un monitoring en temps réel.

#### 3.2 Nouvelle Architecture proposée

Pour répondre aux exigences de contrôle autonome et de transmission magnétique, nous avons choisi de nouveaux composants, tout en conservant le montage de propulsion (ESC

et propulseurs T200).

### 3.2.1 Unité de Calcul et de Contrôle

Le micro-contrôleur Arduino Uno a été remplacé par une Raspberry Pi 4 combinée au contrôleur de vol Navigator de BlueRobotics [5], visible sur la Figure 3. Ce choix a été fait pour plusieurs raisons :

- **Compatibilité ROS 2** : La carte Raspberry Pi 4 permet d'embarquer BlueOS [6] (basé sur Linux) qui peut exécuter nos nœuds ROS 2, notamment pour le contrôle, la trajectoire autonome et les moteurs.
- **Capteurs intégrés** : Le Navigator Flight Controller intègre une IMU et des magnétomètres qui permettent de stabiliser le robot et de connaître son cap.
- **Écosystème BlueOS** : Ce logiciel facilite la gestion des drivers et du pilotage via Wi-Fi, ce qui évite la nécessité d'une manette et d'un système radio.

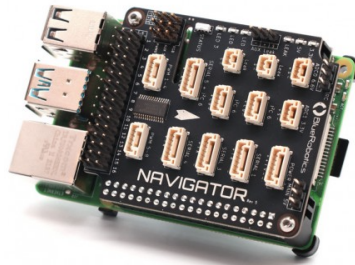


FIGURE 3 – Contrôleur de vol Navigator monté sur la Raspberry Pi 4 [5]

### 3.2.2 Énergie

Pour la batterie, nous avons conservé la technologie LiPo 3S (11.1V). Du fait de l'absence d'informations sur le taux de décharge de l'ancienne batterie, nous avons choisi une batterie adaptée à notre robot : une LiPo 3S de 5000 mAh avec un taux de décharge de 60C [7] (voir Figure 4). L'alimentation de l'électronique de commande est assurée par un régulateur BEC 5V 6A [8] (voir Figure 5).

- **Puissance** : L'ensemble des 4 propulseurs T200 consomment au maximum 128 A. Or le taux de décharge de 60C permet une décharge de courant de  $5\text{ Ah} \times 60 = 300\text{ A}$ , ce qui offre une marge de sécurité importante et évite les chutes de tension.
- **Autonomie** : La capacité de 5000 mAh offre une autonomie estimée entre 15 et 20 minutes lors d'une mission, ce qui est suffisant pour des essais en bassin.



FIGURE 4 – Batterie LiPo 3S 60C [7]



FIGURE 5 – Régulateur BEC 5V 6A [8]

### 3.2.3 Actionnement et Transmission

Enfin, comme énoncé dans notre cahier des charges, un des objectifs de notre projet est d'étudier la transmission magnétique. Ainsi, le servomoteur a été déplacé à l'intérieur du tube étanche. De ce fait, nous avons changé le modèle ROVMaker par un servomoteur numérique SER0067 High Torque (35 kg.cm) [9] (Figure 6). Cette intégration permet de réduire les frottements et de supprimer les risques de fuites liés à l'arbre moteur traversant.



FIGURE 6 – Servomoteur SER0067 avec encodeur magnétique

## 4 Conception et Amélioration Mécanique

Le précédent prototype du robot PlaSMAR possédait une mécanique fonctionnelle mais améliorable, notamment au niveau de la rotation de l'anneau des propulseurs avant qui se faisait par un servomoteur placé à l'extérieur du corps du robot.

Nous avons alors étudié divers moyens d'améliorer ce prototype, et en avons sorti deux versions avec des systèmes de rotation de l'anneau des propulseurs avants différents.

### 4.1 Modèle à transmission mécanique

Nous avons, pour la première version, conservé le même système de rotation de l'anneau (un servomoteur placé à l'extérieur du corps du robot), en changeant de servomoteur et en adaptant son support (voir figure 7).

Nous avons également fait des supports internes (voir figure 7) afin de pouvoir y disposer toute l'électronique du robot. Nous nous sommes pour cela basés sur des supports, trouvés sur le site **bluerobotics** [10], que nous avons adaptés à notre besoin.

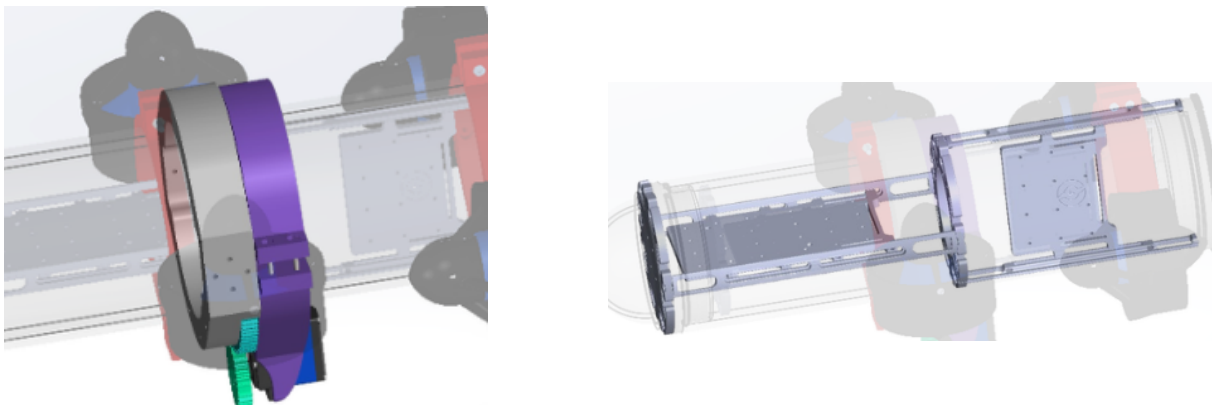


FIGURE 7 – Support nouveau servomoteur (gauche), Support électronique interne (droite)

Nous avons également refait le système d'engrenage liant le servomoteur et l'anneau en rotation, afin de l'adapter à nos besoins.

Le servomoteur retenu n'est cependant pas étanche. Ce choix est dû au fait qu'il était initialement prévu de l'intégrer à l'intérieur du robot pour transmettre magnétiquement la rotation à l'anneau extérieur (solution que nous présenterons dans la partie suivante).

#### 4.1.1 Système final

Cette première solution est ainsi une version améliorée du prototype que l'on avait déjà (voir figure 8). Elle permet de comparer les capacités de ce système avec le système à transmission magnétique.

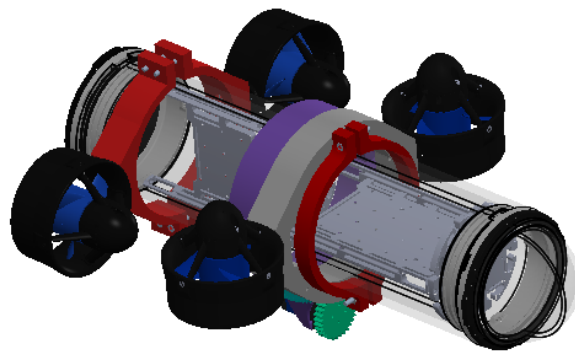


FIGURE 8 – Assemblage à transmission mécanique

## 4.2 Modèle à transmission magnétique

Nous avons, pour la seconde version, changé le modèle de rotation de l'anneau en utilisant un système de transmission magnétique entre un anneau interne et un anneau externe au corps du robot PlaSMAR. Cela afin de pouvoir placer le servomoteur à l'intérieur du robot.

Pour faire tourner l'anneau interne, nous avons repris le principe du système mécanique et l'avons adapté en le faisant passer d'un système externe à un système interne. Nous avons, dans un même temps, adapté l'anneau extérieur et les systèmes de maintien des anneaux pour les limiter à un seul degré de liberté (voir figure 9).

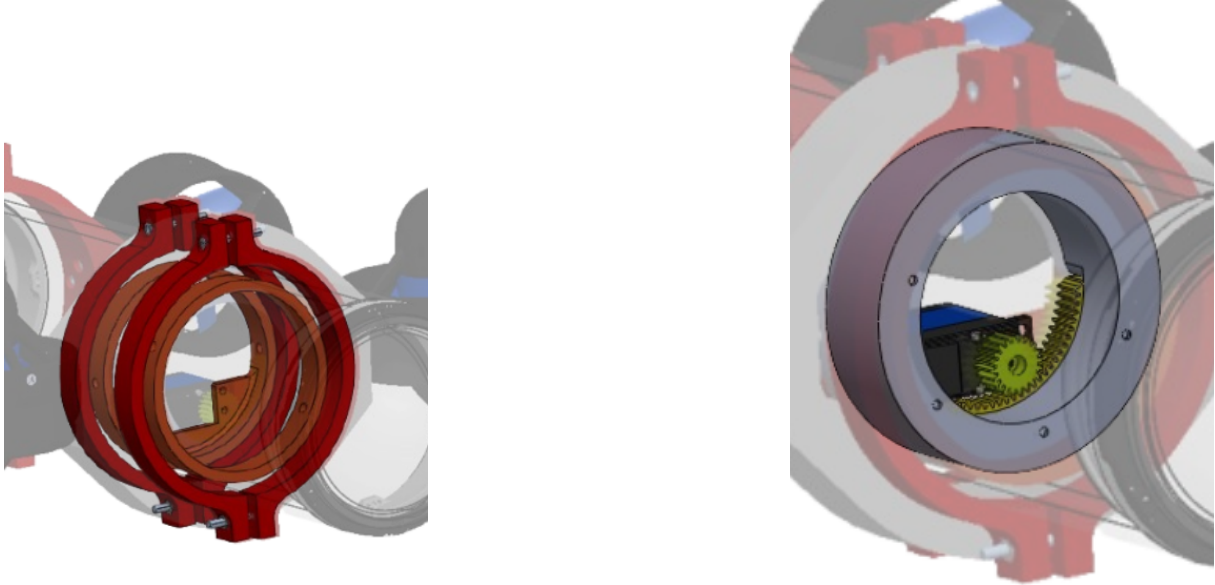


FIGURE 9 – Système de maintien (gauche), Système de transmission interne (droite)

Cette solution supprime tout problème d'étanchéité vis-à-vis du servomoteur, nous avons changé ce dernier par un modèle moins cher et non étanche. Notre choix s'est porté sur le SER0067 de DFROBOT, comportant des caractéristiques techniques similaires à celles du servomoteur du prototype précédent.

#### 4.2.1 Estimation du couple de transmission magnétique minimal nécessaire

Nous avons dans un premier temps estimé le couple de transmission nécessaire avec la formule suivante :

$$C_{min} = \frac{Z.C_s}{I_c}$$

$C_s = 3.5$  : Couple fourni par le servo (N.m)

$Z = \frac{z'}{z}$  : Rapport de transmission des engrenages

$z' = 60$  : Nombre de dents de l'engrenage intérieur

$z = 80$  : Nombre de dents de l'engrenage extérieur

$I_c = 0,7$  : Indice de confiance (marge pour  $C_{min}$ )

Nous avons ainsi obtenu qu'il nous fallait avoir un couple de transmission magnétique minimal :  $C_{min} \approx 6.6 N.m$ .

#### 4.2.2 Calculs du couple de transmission magnétique

Pour le type d'aimants à utiliser, notre choix s'est porté sur des aimants néodyme-fer-bore NdFeB (N42 ou N45) qui offrent une bonne inductance rémanente dans l'environnement du robot.

Pour le dimensionnement des aimants, nous avons basé nos calculs de transmission de

couple entre les deux anneaux magnétiques sur l'article de Thierry LUBIN [11]. En adaptant à notre besoin, nous avons ainsi obtenu la formule suivante :

$$C = \frac{[R_1+R_2]^2}{\pi^2} \cdot B_r^2 \cdot L \cdot \sin^2\left(\frac{\pi}{2} \cdot \alpha\right) \cdot \frac{sh^2\left(\frac{R_1+R_2}{2} \cdot p \cdot h\right)}{sh([R_1+R_2] \cdot [\frac{e}{2}+h] \cdot p)}$$

$B_r = 1,25 \cdot 10^4$  : Inductance rémanente aimants NdFeB  
 $R_1 = 52,65 \cdot 10^{-3} - \frac{h}{2}$  : Rayon moyen anneau intérieur (m)  
 $R_2 = 57,35 \cdot 10^{-3} + \frac{h}{2}$  : Rayon moyen anneau extérieur (m)  
 $L = 50 \cdot 10^{-3}$  : Longueur aimants(m)  
 $h = 10 \cdot 10^{-3}$  : Epaisseur aimants(m)  
 $e = 8,7 \cdot 10^{-3}$  : entrefer (m) (espace entre les anneaux)  
 $\alpha = 45$  : Ouverture des aimants (°)  
 $p = 4$  : Nombre de paires d'aimants par anneau (16 en tout)

Après quelques tests nous avons conclu que le nombre optimal de paire d'aimants par anneau est de 4 (donc 16 aimants au total).

Le modèle alors obtenu, bien qu'assez fiable, néglige certains paramètres, notamment l'influence de la paroi entre les deux anneaux. C'est pourquoi nous mettons un indice de confiance  $I_c$  de 70% pour l'estimation du couple de transmission nécessaire.

#### 4.2.3 Calculs du poids des aimants

Afin de s'assurer que le système soit réaliste, nous avons calculé le poids des aimants pour un tel système :

$$P = d \cdot V$$

$d = 7,5$  : densité NdFeB ( $g \cdot cm^{-3}$ )  
 $V = \pi \cdot L [R_{1,ext}^2 - R_{1,int}^2 - R_{2,ext}^2 + R_{2,int}^2] = 345,57$  : Volume total des aimants ( $cm^3$ )

Nous obtenons ainsi un total de  $P \approx 2.6kg$  pour notre système, ce qui reste acceptable pour le robot PlaSMAR.

#### 4.2.4 Système final

On obtient ainsi un système à double anneau magnétique de 16 aimants néodyme (voir figure 10), qui nous permet de faire tourner les propulseurs autour d'un axe tout en gardant les éléments sensibles à l'intérieur du corps du robot.

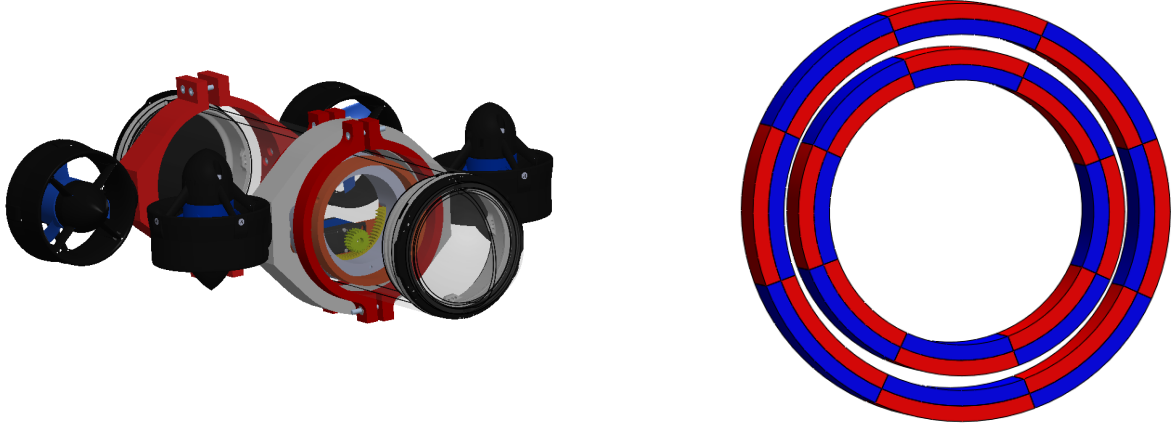


FIGURE 10 – Assemblage à transmission magnétique (gauche), Anneaux magnétiques (droite)

### 4.3 État actuel

N'ayant pas d'aimants convenables, la solution choisie actuellement est le système mécanique avec le servomoteur à l'extérieur, de sorte à pouvoir effectuer différents tests.

Les aimants nécessaires pour la transmission magnétique étant très spécifiques, il est difficile d'en obtenir des convenables. Nous avons cependant trouvé des fournisseurs pour des aimants similaires [12] à ce que nous voulions et pour des aimants fabriqués sur mesure (voir Annexe A.3).

Une fois les aimants choisis, il faudra adapter les 2 anneaux les accueillant en creusant les espaces nécessaires.

## 5 Modélisation et simulation du robot

Avant de déployer le code sur le robot PlaSMAR, nous avons développé un environnement de simulation complet sous ROS 2. L'objectif de cette simulation est triple : modéliser la physique du robot (cinématique, dynamique et comportement de l'anneau rotatif), valider l'architecture logicielle des nœuds développés, et obtenir une visualisation des résultats via Foxglove [13].

Le développement a suivi une approche incrémentale, débutant par un modèle cinématique simplifié pour évoluer vers une simulation complète et réaliste du comportement du robot.

### 5.1 Première version : Validation architecturale (Proof of Concept)

Ce premier modèle de simulation est basé sur la validation de notre architecture logicielle et de la logique de navigation. Il s'appuie sur un modèle cinématique idéal où le robot exécutait instantanément les consignes de vitesse sans inertie ni frottements, implémenté dans le nœud `plasmар_physique` :

$$P_{k+1} = P_k + V_{cmd} \cdot dt \quad (1)$$

Cette version simplifiée nous a permis de valider trois axes majeurs :

- **L’infrastructure ROS** : La bonne communication entre les nœuds, la configuration de l’environnement de simulation combinant Docker [14] et Foxglove, et la cohérence des TF Broadcasting (transformations de repères) entre `map` et `base_link` en temps réel.
- **Contrôle** : Le bon fonctionnement du nœud `controller_node`, qui écoute les commandes clavier, envoie les vitesses au robot et gère le passage en mode autonome.
- **Génération de la trajectoire** : La validation du nœud de planification `plasmар_trajectory_node`. L’algorithme générait correctement les points de passage pour le balayage de zone et calculait les consignes de cap orientées vers la cible.

## 5.2 Deuxième version : Intégration du Modèle Physique Complet

Après validation du premier modèle, nous avons implémenté la physique complète du robot en nous appuyant sur le modèle mathématique décrit dans la thèse de Loïck Degorre [15].

Cette modélisation a été implémentée dans la librairie `plasmар_maths`, développée en Python avec la bibliothèque de calcul numérique NumPy [16], pour faciliter la réutilisation des fonctions physiques dans d’autres nœuds (simulation, contrôle).

Le modèle d’état du robot repose sur les vecteurs cinématiques suivants :

- $\eta$  : Position et orientation dans le repère Monde.
- $\nu$  : Vitesses linéaires et angulaires dans le repère Robot.

Le lien cinématique est assuré par l’équation  $\dot{\eta} = J(\eta)\nu$ , où la matrice de transformation  $J(\eta)$  est implémentée dans la librairie et recalculée à chaque pas de temps pour suivre l’orientation du robot.

Le comportement du robot PlaSMAR est régi par la seconde loi de Newton pour un corps immergé, telle que formalisée par Fossen [17] :

$$\tau = M\dot{\nu} + C(\nu)\nu + D(\nu)\nu + g(\eta) \quad (2)$$

Chaque terme a été implémenté séparément dans la classe `PlasmарModel` afin de conserver la structure mathématique du modèle tout en permettant une exécution temps réel.

### 5.2.1 Masse et Inertie $M$

Cette matrice combine la masse du corps rigide ( $M_b$ ) et les masses ajoutées ( $M_a$ ) dues au déplacement du fluide :  $M = M_b + M_a$ .

Dans notre librairie, nous avons d’abord implémenté la fonction utilitaire `S(v)` générant la matrice antisymétrique (*Skew-symmetric Matrix*).

La méthode `get_M` utilise ensuite les paramètres physiques constants (masse, tenseur d’inertie, centre de gravité) pour construire et sommer les matrices  $M_b$  et  $M_a$ .

### 5.2.2 Effet de Coriolis $C$

Cette matrice modélise les forces fictives (centrifuges et gyroscopiques) dues à la rotation du repère mobile. Contrairement à la masse, elle dépend de l'état du robot et combine les effets du corps rigide et de la masse ajoutée :  $C(\nu) = C_b(\nu) + C_a(\nu)$

La matrice est recalculée à chaque pas de temps via la fonction `get_C` de la librairie `plasmar_maths`.

La fonction décompose d'abord le vecteur de vitesse global  $\nu$  en deux sous-vecteurs : linéaire ( $\nu_1$ ) et angulaire ( $\nu_2$ ). Elle utilise ensuite les paramètres physiques ( $m, I_o, PG, S$ ) et les coefficients hydrodynamiques (notamment les termes croisés comme  $X_{\dot{u}}u$ ) pour construire les matrices  $C_b$  et  $C_a$  avant de renvoyer leur somme.

### 5.2.3 Amortissement Hydrodynamique $D$

Cette matrice modélise les forces de frottement (traînée) qui s'opposent au mouvement du robot dans l'eau. Elle combine deux phénomènes : l'amortissement linéaire (prépondérant à basse vitesse) et l'amortissement quadratique (dominant à haute vitesse).

La matrice est recalculée à chaque pas de temps via la fonction `get_D` de `plasmar_maths`.

Le code récupère les paramètres physiques linéaires et quadratiques et applique, pour chaque axe, la formule : `abs(Xu) + abs(Xuu) * abs(nu)`, ce qui garantit que la force soit toujours opposée au mouvement. La fonction retourne ensuite une matrice diagonale, supposant que les couplages de frottement entre les axes sont négligeables pour la forme cylindrique du robot.

### 5.2.4 Gravité et Stabilité $g$

Enfin, ce vecteur regroupe les forces de gravité (Poids) et la poussée d'Archimède. La stabilité passive est assurée par la distance entre le centre de gravité ( $CG$ ) et le centre de carène ( $CB$ ). Ce décalage crée un couple de rappel hydrostatique qui ramène naturellement le robot à l'horizontale (annulation du Roulis et du Tangage).

La fonction `get_g` projette le vecteur gravité du repère Monde vers le repère Robot en utilisant les angles d'Euler (matrice de rotation). Elle calcule ensuite les couples de redressement nécessaires pour stabiliser le robot, proportionnels au sinus de l'angle d'inclinaison.

### 5.2.5 Loi de Commande

La dernière modification apportée à cette version est l'implémentation d'une loi de commande, reposant sur la linéarisation par retour d'état.

L'implémentation de cette commande se fait en deux étapes :

#### Calcul de la consigne de vitesse

Nous définissons d'abord une vitesse de consigne  $\nu^*$  qui permet de corriger l'erreur de position  $e_\eta = \eta^* - \eta$ . Le modèle cinématique inverse nous permet d'écrire :  $\nu^* = J^{-1}(\eta)(\dot{\eta}^* + K_p e_\eta)$

Cela garantit que l'erreur de position décroît exponentiellement vers 0, si le robot se rapproche de cette vitesse.

### Calcul des couples

Ensuite, le contrôleur calcule les efforts  $\tau$  nécessaires pour que le robot atteigne cette vitesse cible. La dynamique du robot nous donne la loi de commande :  $\tau = M(\dot{\nu}^* + K_v e_\nu) + C(\nu)\nu + D(\nu)\nu + g(\eta)$  Cette équation compense les non-linéarités (Coriolis, Amortissement, Gravité) pour imposer un comportement linéaire au système.

#### 5.2.6 Implémentation logicielle de la simulation

Afin de mettre en œuvre les modèles mathématiques et la loi de commande détaillés ci-dessus, notre architecture s'appuie sur la répartition des tâches entre plusieurs nœuds ROS 2 :

##### Nœud de simulation (plasmар\_physique) :

Le nœud `plasmар_physique` gère l'exécution en temps réel de la physique du robot. Il agit comme un jumeau numérique du robot : il reçoit les commandes de poussée, gère l'allocation des propulseurs (fonction `get_motor_allocation`) et la dynamique temporelle de l'anneau (fonction `cmd_callback`). Dans la fonction `update_physics`, il met à jour les efforts et calcule la nouvelle odométrie à chaque pas de temps.

##### Nœud de suivi de trajectoire (traj\_tracking\_node) :

La loi de commande est intégrée dans ce nœud. Ce choix a été fait pour distinguer le calcul de l'asservissement physique de la logique de haut niveau gérée par le contrôleur. Il génère également la trajectoire que doit suivre le robot lors du mode autonome (fonctions `generate_seabed_scan` et `build_mission_path`), l'affiche dans Foxglove et gère le suivi de la cible (fonctions `update_virtual_target` et `setup_segment`).

##### Nœud du contrôleur (controller\_node) :

Le nœud `controller_node` agit comme le superviseur du système. Il gère la machine à états du robot : en mode manuel, il convertit les touches clavier en consignes de vitesse, en mode autonome, il délègue le pilotage au nœud de suivi de trajectoire. Enfin, il intègre une sécurité en permettant une reprise en main manuelle immédiate par l'opérateur en cas d'imprévu.

#### 5.2.7 Limitations

Bien que théoriquement rigoureuse, l'implémentation directe de ce modèle physique s'est avérée incomplète en simulation. Les résultats expérimentaux ont révélé un défaut de convergence : en mode autonome, le robot ne ralliait pas la trajectoire de référence mais reproduisait sa **forme** localement avec un décalage constant.

Ce comportement a mis en évidence la sensibilité de la loi de commande aux erreurs de modélisation et aux limites physiques des actionneurs (saturation). Ainsi, nous avons développé une troisième version (Banc de Test) afin de valider la physique pas à pas et d'affiner les paramètres.

### 5.3 Troisième version : Banc de Test et Monitoring

Face au défaut de convergence constaté lors de la version précédente, nous avons développé un banc de test. L'objectif était d'isoler le comportement physique du robot pour corriger les actionneurs avant de se concentrer sur le guidage autonome. Le développement s'est déroulé en trois étapes.

#### 5.3.1 Protocoles de Test

Pour analyser le comportement dynamique du robot, le nœud `Test_unit_node` a été implémenté. Il remplace le contrôleur afin d'assurer les mouvements en boucle ouverte et de valider la physique du robot.

Il propose quatre tests différents :

- **Test de stabilité passive** : Les propulseurs sont désactivés. Le robot est initialisé avec une gîte importante (Pitch et Roll de 90°) pour vérifier que le couple de rappel hydrostatique ( $g$ ) ramène naturellement le robot à l'horizontale.
- **Test d'avance** : Une force de 20 N (échelon) est appliquée sur l'axe X. L'absence de dérive latérale valide le découplage des axes et la matrice d'amortissement  $D$ .
- **Test latéral** : Une force de 20 N (échelon) est appliquée sur l'axe Y. Ce test valide la dynamique de l'anneau rotatif et vérifie que le robot se déplace latéralement sans rotation.
- **Séquence complète** : Le robot exécute un enchaînement automatisé (Avance → Latéral → Plongée → Avance → Remontée) afin de valider la superposition des forces.

#### 5.3.2 Monitoring

Afin de visualiser de façon technique les protocoles de test précédents, nous avons développé le nœud de monitoring `plot_node`. Il permet de générer les courbes de performances du robot comme : sa trajectoire 3D, ses trajectoires 2D, la stabilité angulaire, l'orientation des propulseurs, les vitesses de rotation (RPM) ainsi que les efforts de commande.

La particularité de ce monitoring est que nous ne pouvons pas étudier les courbes en direct car la simulation tourne dans le conteneur Docker. Ainsi, le nœud utilise *Matplotlib* [18] pour sauvegarder les courbes une fois les tests finis.

#### 5.3.3 Caractérisation et modélisation des actionneurs

Les premiers résultats ont montré des réponses trop rapides (échelons parfaits) lors de l'envoi de la force, ne reflétant pas la réalité. Ainsi, nous avons affiné la modélisation.

##### Dynamique des propulseurs

Nous avons modélisé les moteurs brushless présents dans les T200 par un système du premier ordre, discrétisé par la méthode d'Euler explicite :

$$F_k = \alpha F_{cmd} + (1 - \alpha) F_{k-1}, \quad \text{avec } \tau \approx 0.15s \quad (3)$$

Ainsi, la réponse à un échelon de commande n'est plus un rectangle parfait mais un trapèze, qui reflète le temps de montée du moteur.

## Lookup Table de chez BlueRobotics

Nous avons intégré une Lookup Table issue des données constructeur BlueRobotics [3] pour compléter la dynamique. Celle-ci permet de convertir la force demandée en commande RPM précise selon le mouvement avant ou arrière, de gérer la saturation physique à  $\pm 40$  N et de prendre en compte les zones mortes à basses vitesses.

### 5.3.4 Modélisation de l'anneau et allocation

Selon la thèse de Loïck Degorre [15], l'angle de l'anneau rotatif peut être approximé par :

$$\alpha_c = \arctan2(Z_c, Y_c) \quad (4)$$

Avec  $Z_c$  et  $Y_c$ , les forces de commande en *sway* (latéral) et *heave* (vertical).

Cet angle joue un rôle important dans l'allocation des propulseurs. La matrice de configuration (TCM), notée  $B$ , est recalculée à chaque pas de temps. Comme le système est sous-actionné ( $6 \times 4$ ) et que  $B$  n'est pas carrée, nous utilisons sa pseudo-inverse  $B^+$  pour déterminer les consignes de poussée  $u = B^+ \cdot \tau_c$ , avec :

$$B^+ = \begin{bmatrix} 1/2 & 0 & 0 & 0 & 0 & -1/2R \\ 1/2 & 0 & 0 & 0 & 0 & 1/2R \\ 0 & \frac{\sin \alpha}{2} & -\frac{\cos \alpha}{2} & -\frac{1}{2R} & 0 & 0 \\ 0 & \frac{\sin \alpha}{2} & -\frac{\cos \alpha}{2} & \frac{1}{2R} & 0 & 0 \end{bmatrix} \quad (5)$$

### 5.3.5 Résultats

Les tests de stabilité passive et de séquence complète (voir Annexes A.2) ont validé le comportement global. Cependant, l'analyse s'est concentrée sur les phases critiques (Avance et Latéral) présentées ci-dessous :

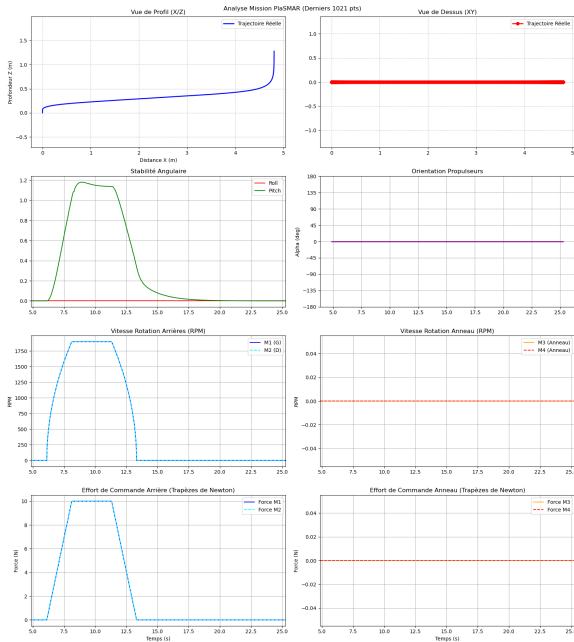


FIGURE 11 – Courbes de performances du test Avancer

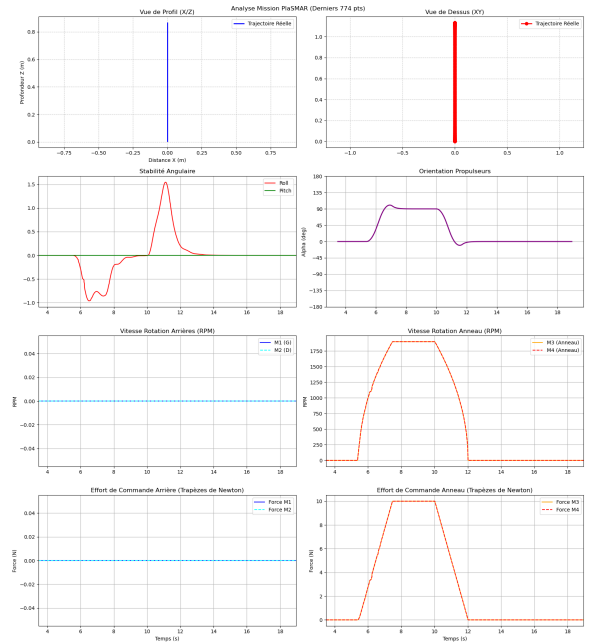


FIGURE 12 – Courbes de performances du test déplacement latéral

## Analyse des courbes

- **Sur la Figure 11 (Avance) :** On observe une montée progressive des moteurs arrière, ce qui valide notre caractérisation des propulseurs. On remarque aussi que seuls les moteurs arrière sont activés pour faire avancer le robot. De plus, la vue de dessus montre que la trajectoire suivie est bien droite. La vue de profil est légèrement courbée, ce qui est dû à la poussée d'Archimède qui fait remonter le robot.
- **Sur la Figure 12 (Latéral) :** On constate que le robot maintient bien son cap à  $0^\circ$  et que sa trajectoire vue de dessus est bien latérale. De plus, l'anneau s'oriente correctement pour permettre l'utilisation des propulseurs avant. Cela valide donc la matrice d'allocation des propulseurs.

## 5.4 Version Finale : Suivi de trajectoire et mode autonome

Suite à la validation des modèles physiques par le Banc de Test, nous avons implémenté la version finale. L'objectif était de passer de la validation unitaire à une simulation complète en boucle fermée, capable de corriger le défaut de convergence et d'assurer un balayage fluide de la zone.

Les améliorations ont été implémentées dans le nœud `traj_tracking_node`.

### 5.4.1 Séquencement de la mission

Afin d'éviter la saturation des actionneurs en essayant d'avancer, de tourner et de plonger en même temps, nous avons décomposé la mission en plusieurs étapes :

- **Plongée :** Le robot descend verticalement à la profondeur cible ( $z = -1.5$  m) par le biais d'un asservissement, avant tout déplacement horizontal.
- **Approche :** Une fois à la bonne profondeur, il rejoint le départ de la trajectoire.
- **Balayage :** Il exécute la trajectoire prédéfinie.

Pour le balayage, nous avons exploité la capacité du robot à se déplacer latéralement sans changer de cap. Pour cela, le code distingue deux types de segments :

- **Le segment normal :** Le robot oriente son nez vers l'avant.
- **Le segment latéral :** Le robot verrouille son cap et se déplace selon l'axe Y.

**Gestion des conflits d'allocation :** Le maintien de la profondeur et le déplacement latéral utilisent les mêmes propulseurs verticaux. Pour éviter les à-coups, nous avons diminué la consigne de vitesse latérale à  $0.1 \text{ m} \cdot \text{s}^{-1}$ .

### 5.4.2 Amélioration de la loi de commande

Pour garantir la convergence, nous avons amélioré la loi de commande :

- **Ajout d'une action intégrale :** Nous avons ajouté un terme intégral pour annuler l'erreur statique, due aux courants :

$$u = K \cdot e + K_i \int e dt \quad (6)$$

Pour éviter un dépassement excessif, nous avons ajouté une saturation de l'intégrale à  $\pm 5$ .

- **Réglage de la saturation :** Suite aux résultats du banc de test, nous avons remarqué que la saturation de 40 N pour un propulseur était en réalité appliquée globalement. Nous l'avons donc élargie à 80 N.

Ces modifications assurent que le robot suive la trajectoire définie avec précision sans diverger.

### 5.4.3 Validation de l'architecture (RQT Graph)

Le graphe des flux de données final de la simulation (voir Figure 13) confirme la structure en boucle fermée du système. Le nœud `controller` sert d'interface à l'utilisateur et déclenche la mission via le topic `/plasmar/auto_trigger`. Le `trajectory_node` génère alors les consignes envoyées au moteur physique, qui renvoie en retour l'odométrie pour réaliser l'asservissement.

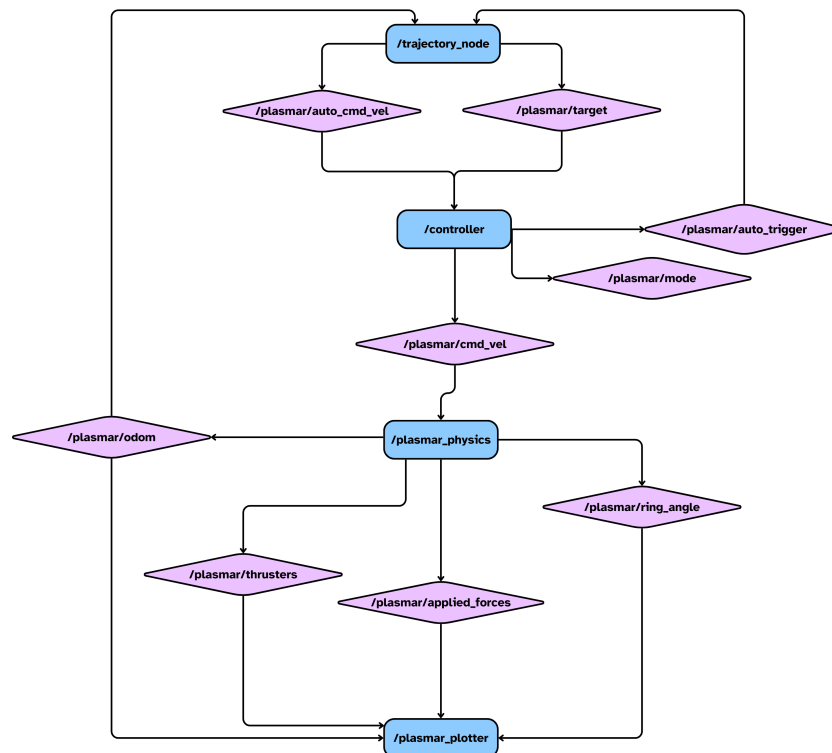


FIGURE 13 – Architecture logicielle ROS 2 simplifiée

### 5.4.4 Résultats Simulation

La simulation valide le bon fonctionnement du mode autonome. La Figure 14 démontre que le robot parvient à suivre la trajectoire désirée, en effectuant ici un balayage complet à une profondeur constante de -1,5 m.

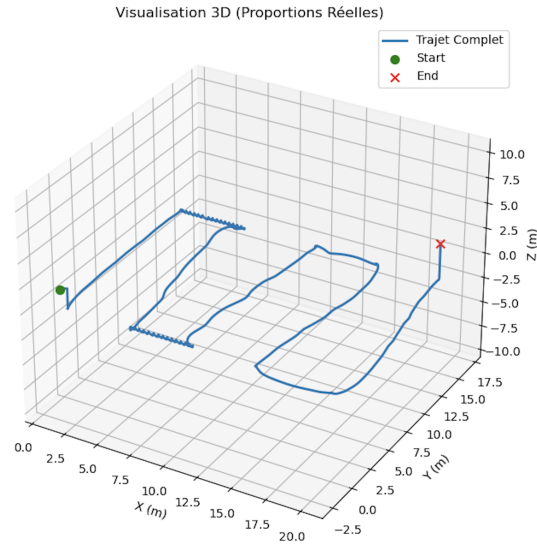


FIGURE 14 – Visualisation 3D de la trajectoire du mode autonome

Les captures d'écran issues de l'interface Foxglove (Figures 15, 16 et 17) illustrent le comportement dynamique du robot lors des différentes phases de sa mission : ralliement du point de départ, avance et déplacement latéral.

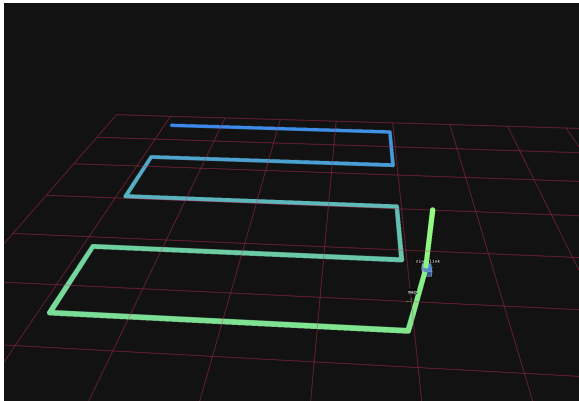


FIGURE 15 – Descente du robot pour se mettre à la bonne profondeur

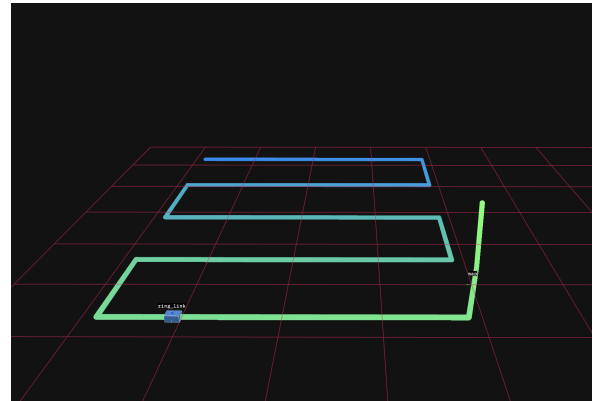


FIGURE 16 – Avance du robot

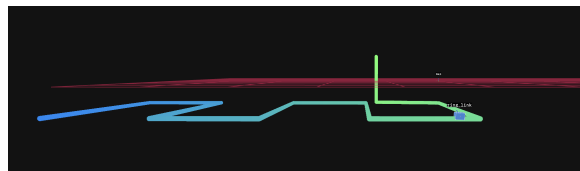


FIGURE 17 – Déplacement latéral du robot

Cette dernière version offre une modélisation fidèle du comportement dynamique du robot PlaSMAR. Après plusieurs versions dont un banc de test, cette simulation complète confirme la robustesse de l'architecture ROS 2 (communication inter-nœuds) ainsi que la pertinence des modèles physiques implémentés, garantissant des résultats conformes aux

exigences de la mission. Il est important de noter que cette simulation reste théorique et que la réalité introduit d'autres facteurs importants, tels que l'absence de signal GPS sous l'eau ou la nécessité de s'adapter aux véritables capteurs embarqués.

## 6 Réalisation et Intégration

Cette section a pour but de décrire comment le robot PlaSMAR a été réalisé et intégré. On met particulièrement l'accent sur l'intégration électronique des propulseurs et du servomoteur, la configuration du contrôleur de vol Navigator sous BlueOS, ainsi que le déploiement des nœuds ROS2 qui permettent de contrôler le robot via MAVROS. Cette section explique aussi comment les consignes de mouvement sont converties en signaux PWM que les moteurs peuvent utiliser.

On soulignera également les défis techniques rencontrés lors de l'intégration, notamment ceux liés à l'environnement BlueOS et à la communication entre les différents nœuds ROS2, en expliquant comment ces obstacles ont influencé les choix d'implémentation qui ont été faits.

### 6.1 Intégration matérielle et câblage

L'intégration matérielle du robot PlaSMAR repose sur une architecture combinant une unité de calcul embarquée, un contrôleur de vol, des actionneurs électriques et une alimentation adaptée aux contraintes sous-marines.

Les composants utilisés pour cette intégration sont ceux décrits dans les sections précédentes, à savoir la Raspberry Pi 4 associée au Flight Controller Navigator de BlueRobotics, les quatre propulseurs T200 avec leurs ESC dédiés, le servomoteur assurant la rotation de l'anneau des propulseurs avant, ainsi que la batterie LiPo 3S fournissant l'énergie du système. Un schéma global du câblage et de l'architecture matérielle est présenté en figure 18.

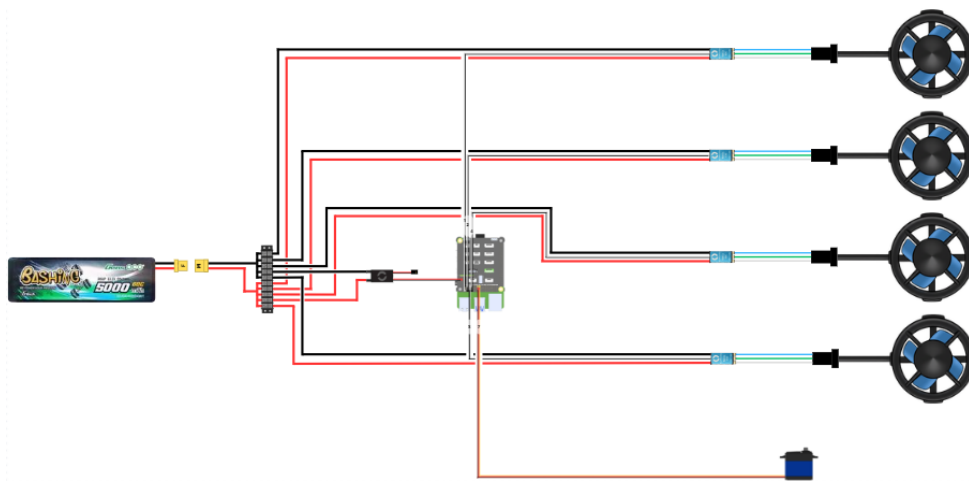


FIGURE 18 – Schéma global de l'architecture matérielle du robot PlaSMAR

### 6.1.1 Rôle des ESC et distribution de puissance

Les propulseurs T200 étant des moteurs brushless triphasés, ils nécessitent l'utilisation de contrôleurs électroniques de vitesse (ESC) pour fonctionner. Les ESC assurent la conversion de la tension continue fournie par la batterie LiPo 3S en signaux triphasés adaptés aux moteurs, tout en interprétant les signaux PWM de type RC envoyés par le Flight Navigator afin de définir la vitesse et le sens de rotation des propulseurs.

Les ESC sont alimentés directement par la batterie, tandis que leurs signaux de commande sont reliés au contrôleur de vol. Le câblage et les recommandations de branchement ont été réalisés en s'appuyant sur la documentation constructeur des propulseurs T200 fournie par Blue Robotics [3].

### 6.1.2 Choix des branchements sur le Flight Navigator

Le Flight Navigator dispose de sorties PWM dédiées, accessibles via les ports AUX. Dans notre configuration, les actionneurs sont connectés comme suit :

- les deux propulseurs arrière horizontaux sont connectés aux sorties AUX1 et AUX2 ;
- les deux propulseurs avant, montés sur l'anneau rotatif, sont connectés aux sorties AUX3 et AUX4 ;
- le servomoteur de rotation de l'anneau est connecté à la sortie AUX7.

Les informations relatives au brochage, aux fonctionnalités des sorties AUX et aux recommandations de câblage ont été obtenues à partir de la documentation officielle du Flight Navigator fournie par BlueRobotics [19].

Une vue détaillée du Flight Navigator et des connexions utilisées est présentée en figure 19.

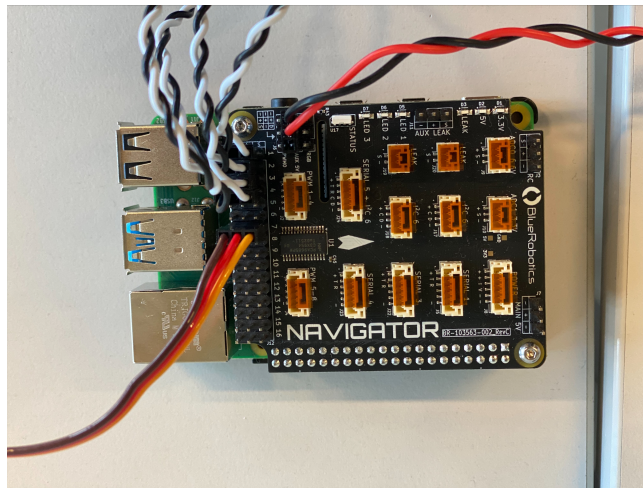


FIGURE 19 – Flight Navigator Blue Robotics et répartition des connexions

## 6.2 Configuration de BlueOS et du contrôleur de vol

Le système d'exploitation *BlueOS* était déjà préinstallé sur la carte Raspberry Pi intégrée au Flight Navigator.

### 6.2.1 Connexion à l'interface BlueOS

La connexion à l'interface web de BlueOS a été réalisée via une liaison Wi-Fi conforme aux recommandations de BlueRobotics [20]. D'après la documentation BlueRobotics, la Raspberry Pi embarquée est préconfigurée avec une adresse IP statique 192.168.2.2 et attend une station de contrôle configurée en 192.168.2.1. Dans notre cas, la connexion s'est faite en rejoignant le même réseau Wi-Fi que le robot, puis en accédant à l'interface BlueOS depuis un navigateur web.

### 6.2.2 Ajout de ROS2 et compatibilité système

L'un des objectifs majeurs de ce projet était de déployer des nœuds ROS2 directement sur le robot afin de commander les actionneurs de manière indépendante. Toutefois, la version initiale de BlueOS installée sur la Raspberry Pi reposait sur une architecture 32 bits, incompatible avec les extensions ROS2 disponibles.

Afin de résoudre ce problème de compatibilité, il a été nécessaire de re-flasher entièrement la carte microSD de la Raspberry Pi avec une version 64 bits de BlueOS. Cette opération a été réalisée en suivant la procédure officielle décrite dans la documentation BlueOS [21]. Une fois cette nouvelle version installée, l'extension ROS2 a pu être ajoutée et utilisée sans restriction.

### 6.2.3 Configuration des sorties PWM et mode RC Override

Par défaut, BlueOS configure les moteurs pour un pilotage de type ROV, où les sorties PWM sont directement associées à des degrés de liberté globaux (avance, rotation, plongée). Or, l'objectif de notre projet était de contrôler chaque actionneur individuellement depuis ROS2.

Pour cela, une modification de la configuration a été effectuée dans l'onglet *Vehicle Setup* de BlueOS. Les sorties PWM correspondant aux actionneurs (`Motor1`, `Motor2`, `Motor3`, `Motor4` et `Motor7`) ont été reconfigurées en mode `RCIN`. Ce choix permet d'ignorer la logique de pilotage ROV intégrée et d'autoriser l'envoi de commandes directes via des messages `RC Override` depuis MAVROS et ROS2.

Cette configuration est essentielle pour implémenter des lois de commande personnalisées et pour tester indépendamment chaque moteur et le servomoteur de l'anneau rotatif.

## 6.3 Déploiement et architecture logicielle ROS2

Afin de permettre le pilotage bas niveau des actionneurs du robot PlaSMAR, une architecture logicielle ROS2 a été mise en place au sein de l'environnement BlueOS. L'un des premiers enjeux a été de garantir la persistance des développements, les conteneurs BlueOS étant par défaut non persistants.

Pour résoudre ce problème, un workspace ROS2 dédié a été créé dans le répertoire `persistent_ws` de BlueOS, qui est monté de manière permanente sur la Raspberry Pi. Après la création du workspace, un paquet ROS2 a été généré, puis configuré via les fichiers standards `package.xml` et `setup.py`, avant compilation avec `colcon build`. Cette organisation garantit que les nœuds restent disponibles après chaque redémarrage du système.

L'objectif de cette étape était de valider progressivement la chaîne complète de commande :

*clavier*  $\rightarrow$  *ROS2*  $\rightarrow$  *MAVROS*  $\rightarrow$  *RC override*  $\rightarrow$  *actionneurs*.

Pour cela, plusieurs nœuds ROS2 spécifiques ont été développés afin de tester séparément le servomoteur et chacun des propulseurs T200, avant d'implémenter une logique de commande globale.

### Nœud `rc_servo` : commande du servomoteur

Le nœud `rc_servo_node` est dédié au pilotage du servomoteur assurant la rotation de l'anneau supportant les propulseurs avant (canal RC7). Il écoute le topic `/cmd_vel` et exploite la composante `angular.z` pour générer une commande PWM centrée autour de 1500  $\mu$ s, avec saturation afin de respecter les limites mécaniques du système.

### Fichier de lancement `servo.launch.py`

Un fichier de lancement ROS2 a été développé afin d'automatiser les phases de test. Ce fichier assure :

1. l'armement du contrôleur de vol via le service `/mavros/cmd/arming`;
2. le lancement temporisé du nœud de commande du servomoteur.

Cette temporisation garantit que les commandes PWM ne sont envoyées qu'une fois MAVROS correctement initialisé.

### Nœud `rc_thruster_node` : commande d'un propulseur T200

Le nœud `rc_thruster_node` permet de commander individuellement un propulseur T200 via un canal RC configurable. Il utilise la composante `linear.x` du message `/cmd_vel`, interprétée comme une consigne normalisée comprise entre  $-1$  et  $1$ , convertie ensuite en valeur PWM.

Grâce au paramètre `rc_channel`, ce même nœud a pu être utilisé pour tester successivement chacun des propulseurs connectés aux sorties AUX1 à AUX4 du Flight Navigator. Ces essais ont permis de valider le câblage, le sens de rotation et les plages PWM effectives des moteurs.

L'ensemble des codes est fourni en annexe. (annexe A.1).

### Remarques sur le déploiement ROS2

- Les différents nœuds ont également été testés en utilisant un pilotage clavier. Pour cela, le nœud standard `teleop_twist_keyboard` a été utilisé. Ce nœud n'a pas été lancé depuis BlueOS, mais directement depuis l'ordinateur de contrôle, connecté au même réseau que la Raspberry Pi. Les commandes clavier sont ainsi publiées sur le topic `/cmd_vel`, puis transmises au robot via le réseau.
- Initialement, il avait été envisagé d'utiliser des fichiers `.launch` de type ROS 1. Cependant, en ROS 2, ce format n'est plus nativement supporté et son utilisation devient rapidement complexe. L'utilisation de fichiers `.launch.py` s'est donc imposée comme une solution plus adaptée.

- Le développement des nœuds ROS 2 s’est appuyé sur l’étude de ressources existantes, en particulier le dépôt GitHub [22], qui a servi de référence pour l’intégration de MAVROS, l’utilisation des commandes `RC Override` et la structuration globale de l’architecture logicielle.
- Plus de détails de fonctionnement de chaque nœud sont explicitement documentés dans les commentaires du code source, fourni en annexe pour faciliter la compréhension.

## 6.4 Logique de commande et allocation des actionneurs

Après avoir validé le fonctionnement individuel de chaque actionneur (propulseurs et servomoteur), il a été nécessaire de réfléchir à leur commande conjointe afin de permettre au robot PlaSMAR de se déplacer de manière cohérente.

Pour structurer cette logique de commande, une approche par **tableau de vérité** a été retenue. Cette méthode permet d’associer explicitement chaque action élémentaire du robot à l’état attendu de chacun des actionneurs, tout en conservant une lecture claire et facilement implémentable dans un nœud ROS2.

### 6.4.1 Tableau de vérité des actions élémentaires

Le tableau 1 présente la logique de commande retenue pour les quatre propulseurs T200 et le servomoteur commandant l’orientation de l’arceau.

TABLE 1 – Tableau de vérité pour l’allocation des actionneurs

| Action         | T1 (pin1) | T2 (pin2) | T3 (pin3) | T4 (pin4) | S7 (pin7) | Touche clavier |
|----------------|-----------|-----------|-----------|-----------|-----------|----------------|
| STOP           | 0         | 0         | 0         | 0         | S0        | k              |
| AVANCER        | +         | +         | +         | +         | S0        | i              |
| RECULER        | –         | –         | –         | –         | S0        | ,              |
| TOURNER DROITE | +         | –         | 0         | 0         | S0        | l              |
| TOURNER GAUCHE | –         | +         | 0         | 0         | S0        | j              |
| MONTER         | 0         | 0         | +         | +         | S↑        | t              |
| DESCENDRE      | 0         | 0         | +         | +         | S↓        | b              |

Avec :

- + : poussée positive (PWM > 1500) ;
- – : poussée négative (PWM < 1500) ;
- 0 : poussée nulle (PWM = 1500) ;
- S0 : position neutre du servomoteur ;
- S↑ / S↓ : orientation de l’arceau vers le haut ou vers le bas.

### 6.4.2 Implémentation logicielle et allocation des actionneurs

Le tableau de vérité présenté précédemment a été implémenté dans un nœud ROS2 dédié, nommé `rc_manipuler_robot`. Ce nœud assure la commande globale du robot en jouant le rôle de *mixer logiciel* entre les entrées utilisateur et les actionneurs physiques. Il écoute les messages de type `Twist` publiés sur le topic `/cmd_vel`, issus du contrôle clavier, et

génère les commandes PWM correspondantes pour chacun des actionneurs via le topic `/mavros/rc/override`.

Dans ce nœud, la composante `linear.x` du message `Twist` est utilisée pour piloter l'avance et le recul du robot (surge) en commandant simultanément les quatre propulseurs, la composante `angular.z` permet de générer un couple de rotation autour de l'axe vertical (yaw) par différentiel de poussée entre les propulseurs arrière, et la composante `linear.z` est exploitée pour la variation de profondeur (heave), en combinaison avec la commande du servomoteur pilotant l'orientation de l'arceau supportant les propulseurs avant.

Cette implémentation permet de traduire directement les intentions de l'utilisateur en commandes cohérentes pour l'ensemble des actionneurs, tout en respectant la logique définie par le tableau de vérité et la cinématique spécifique du robot PlaSMAR.

## 6.5 Commande par allocation matricielle des propulseurs

Après une première approche basée sur un tableau de vérité discret, une méthode plus rigoureuse a été implémentée en s'appuyant sur la matrice de configuration des propulseurs (Thrust Configuration Matrix – TCM) notée  $B$ . La formulation théorique de cette matrice et son inversion par pseudo-inverse de Moore–Penrose ont été détaillées dans la section 5.3.4.

### Lien entre les commandes clavier et la matrice $B$

Chaque composante du message agit sur une ligne spécifique de la matrice  $B$  :

- **Avancer / Reculer (touche i / ,)** agit sur `linear.x`  $\rightarrow X_c$  (surge). D'après la première ligne de  $B$ , seuls les deux propulseurs arrière (colonnes 1 et 2) contribuent à l'effort longitudinal. Ils sont donc sollicités de manière symétrique.
- **Tourner (touche j / 1)** agit sur `angular.z`  $\rightarrow N_c$  (moment de lacet). La dernière ligne de  $B$  montre que le moment de yaw dépend uniquement des propulseurs arrière, avec des contributions opposées ( $-R$  et  $+R$ ). Le robot tourne donc par différentiel de poussée arrière.
- **Déplacement latéral (touche J / L)** agit sur `linear.y`  $\rightarrow Y_c$  (sway). D'après la deuxième ligne de  $B$ , seuls les deux propulseurs montés sur l'anneau (colonnes 3 et 4) génèrent une composante latérale via  $\sin(\alpha_c)$ .
- **Monter / Descendre (touche t / b)** agit sur `linear.z`  $\rightarrow Z_c$  (heave). La troisième ligne de  $B$  montre que les propulseurs avant produisent une composante verticale proportionnelle à  $-\cos(\alpha_c)$ .

Ainsi, la structure même de la matrice  $B$  explique la répartition des efforts :

- Les **propulseurs arrière** assurent le **surge** et le **yaw**.
- Les **propulseurs montés sur l'anneau** assurent le **sway**, le **heave** et le **roll**.

Cette approche matricielle présente un avantage important : elle permet de passer d'une logique discrète (tableau de vérité) à une allocation continue et physiquement cohérente, directement dérivée du modèle dynamique du robot. Elle assure ainsi une transition naturelle entre la commande clavier, la modélisation théorique et l'implémentation réelle sur le robot PlaSMAR.

## 7 Conclusion et perspectives

Ce projet marque la transition du robot PlaSMAR vers une plateforme autonome et modulaire.

### 7.1 Synthèse et validation du système

La mise à jour matérielle (avec le Raspberry Pi 4 et le contrôleur Navigator fonctionnant sous BlueOS) a éliminé les restrictions de calcul précédentes, rendant possible l'intégration de ROS2 pour un contrôle précis et individuel des actionneurs.

En parallèle, le développement d'un environnement de simulation complet a permis de valider la nouvelle architecture. Grâce à un modèle dynamique fidèle et une loi de commande robuste par retour d'état, le robot démontre sa capacité à accomplir des missions autonomes complexes, comme le balayage de zone à profondeur constante.

La base logicielle et matérielle étant validée en simulation, le système est désormais prêt pour les essais en bassin.

### 7.2 Pistes d'améliorations

Il reste néanmoins diverses pistes d'amélioration possibles pour le robot PlaSMAR.

#### 7.2.1 Transmission magnétique

Afin de réduire le poids des aimants (par la même occasion leurs coûts), il existe différentes solutions pour augmenter le couple de transmission, dont la segmentation des aimants ou le réseau Halbach [23]. Ce dernier étant complexe, nous n'avons pas eu le temps de l'étudier en profondeur, et son efficacité reste théorique. Néanmoins cela reste une piste à explorer qui pourrait permettre de réduire le poids total des aimants de 30 %, et ainsi passer de 2,6 kg à 1,8 kg d'aimants.

#### 7.2.2 Essais expérimentaux en bassin

Une des étapes critiques à court terme sera les essais en conditions en bassin. Il s'agira de confronter le jumeau numérique au robot physique pour réajuster certains paramètres (notamment les termes d'amortissement linéaire et quadratique) et vérifier la viabilité de la commande d'allocation de l'anneau rotatif face à la résistance de l'eau.

Ces tests permettront de valider définitivement la fiabilité de la transmission mécanique actuelle avant d'investir dans les aimants sur mesure détaillés en annexe.

#### 7.2.3 Autonomie de navigation et fusion de capteurs

La simulation actuelle suppose une odométrie parfaite. Afin de rendre le robot pleinement autonome en milieu sous-marin (sans signal GPS), les travaux futurs devront se concentrer sur l'implémentation d'algorithmes de fusion de données. L'exploitation avancée de la centrale inertielle (IMU) du Navigator couplée à l'intégration potentielle d'un capteur de pression précis ou d'une caméra pour de l'odométrie visuelle, afin de fiabiliser le suivi de trajectoire.

## Références

- [1] Maxime VIEILLOT. *Conception et réalisation d'un robot reconfigurable*. Rapport de stage assistant ingénieur. Encadrants : Loïck Degorre et Lionel Lapierre. ENSTA Bretagne (Campus de Brest), Laboratoire UFR S2I ROB, Juillet 2025.
- [2] BLUERBOTICS. *Basic ESC*. <https://bluerobotics.com/store/thrusters/speed-controllers/besc30-r3/>. [Consulté en février 2026].
- [3] BLUERBOTICS. *T200 Thruster - Technical Specifications & Performance Charts*. <https://bluerobotics.com/store/thrusters/t100-t200-thrusters/t200-thruster-r2-rp/>. [Consulté en février 2026].
- [4] OPEN ROBOTICS. *The Robot Operating System (ROS 2) - Documentation officielle*. <https://www.ros.org/>. [Consulté en février 2026].
- [5] BLUERBOTICS. *Navigator Flight Controller & Raspberry Pi 4 System*. <https://bluerobotics.com/store/comm-control-power/control/navigator/>. [Consulté en février 2026].
- [6] BLUERBOTICS. *BlueOS : Advanced Underwater Vehicle Operating System*. <https://docs.bluerobotics.com/blueos/>. [Consulté en février 2026].
- [7] GENS ACE. *Bashing Series LiPo 3S 11.1V 5000mAh 60C*. Distribué par FlashRC. <https://www.flashrc.com/fr/batteries-voitures-et-bateaux-rc/39524-batterie-gens-ace-bashing-series-lipo-3s-111v-5000mah-60c-prise-xt90-6928493309360.html>. [Consulté en février 2026].
- [8] BLUERBOTICS. *5V 6A Power Module (BEC) - R1*. <https://bluerobotics.com/store/comm-control-power/control/bec-5v6a-r1/>. [Consulté en février 2026].
- [9] DFROBOT. *SER0067 Digital Servo with Magnetic Encoder (3.48 Nm)*. Distribué par Farnell. <https://fr.farnell.com/dfrobot/ser0067/servo-moteur-num-rique-8-4v-1/dp/4457325>. [Consulté en février 2026].
- [10] BLUERBOTICS. *bluerobotics. Watertight Enclosure RAILS* <https://bluerobotics.com/store/watertight-enclosures/locking-series/watertight-enclosure-rails/>. [Consulté en janvier 2026].
- [11] Thierry LUBIN. “Transmission de couple sans contact - Accouplements magnétiques”. In : *Energies* (2025). Disponible sur : <https://www.techniques-ingenieur.fr/base-documentaire/energies-th4/machines-electriques-tournantes-conception-construction-et-commande-42252210/transmission-de-couple-sans-contact-d3565/>.
- [12] MAGNOSPHERE. *Aimant de segment d'arc en néodyme N42*. Aimants Sud. <https://www.magnosphere.fr/segment-arc-magnet-neodyme-n42-sud-80-degrees-interieur-r33-5mm-x-exterieur-r37-5mm/a-1182/> Aimants Nord. <https://www.magnosphere.fr/segment-arc-magnet-neodyme-n42-nord-80-degrees-interieur-r33-5mm-x-exterieur-r37-5mm/a-1180/>. [Consulté en février 2026].
- [13] FOXGLOVE. *Foxglove : Visualization and debugging for robotics*. <https://foxglove.dev/>. [Consulté en février 2026].
- [14] DOCKER INC. *Docker : Accelerated Container Application Development*. <https://www.docker.com/>. [Consulté en février 2026].
- [15] Loïck DEGORRE. “Analyse et commande des robots sous-marins autonomes à propulsion vectorielle reconfigurable”. NNT : 2023ENIB0002. Disponible sur : <https://hal.science/tel-04400492v2>. Thèse de doctorat. École Nationale d'Ingénieurs de Brest (ENIB), 2023.

- [16] Charles R. HARRIS, K. Jarrod MILLMAN et al. “Array programming with NumPy”. In : *Nature* 585.7825 (2020). Disponible sur : <https://numpy.org/>, p. 357-362.
- [17] Thor I. FOSSEN. *Handbook of Marine Craft Hydrodynamics and Motion Control*. 2nd. Code source : <https://github.com/cybergalactic/FossenHandbook>. John Wiley & Sons, 2021.
- [18] J. D. HUNTER. *Matplotlib : A 2D graphics environment*. Computing in Science & Engineering, 9(3), 90-95. <https://matplotlib.org/>. 2007.
- [19] *Navigator Flight Controller*. <https://bluerobotics.com/store/comm-control-power/control/navigator/>. Accessed : 2025.
- [20] *Navigator Software Setup*. <https://bluerobotics.com/learn/navigator-software-setup/>. Accessed : 2025.
- [21] *BlueOS Installation Guide*. <https://blueos.cloud/docs/stable/usage/installation/>. Accessed : 2025.
- [22] Loïck DEGORRE. *blueboat\_ros\_driver*. Dépôt GitHub. 2023. URL : [https://github.com/loickdegorre/blueboat\\_ros\\_driver](https://github.com/loickdegorre/blueboat_ros_driver) (visité le 01/12/2024).
- [23] NEOMAGNETS. *Réseau de Halbach*. <https://neomagnets.net/fr/halbach-array/>. [Consulté en février 2026].

## A Annexes

### A.1 Workspace ROS2 et codes sources

Le code complet du projet est disponible sur le dépôt GitHub suivant : [Dépôt GitHub du projet PlaSMAR](#).

### A.2 Résultats de simulation complémentaires

Cette annexe regroupe les résultats du banc de test ainsi que les visualisations 3D des trajectoires, qui n'ont pas été détaillés dans le rapport.

#### A.2.1 Stabilité passive

L'objectif de ce test est de valider la matrice  $g(\eta)$  et le couple de redressement hydrostatique. Le robot est lâché dans l'eau sans aucune commande moteur.

##### Cas 1

Pour ce premier test, le robot est initialisé à l'horizontale. On vérifie qu'il reste stable (Roll/Pitch  $\approx 0$ ) et qu'il remonte légèrement sous l'effet d'Archimède.

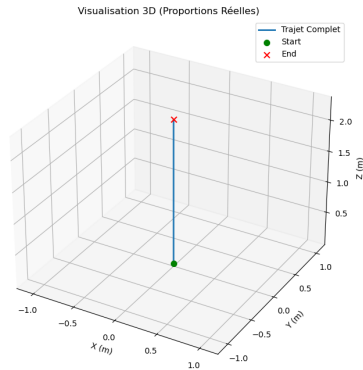


FIGURE 20 – Trajectoire verticale

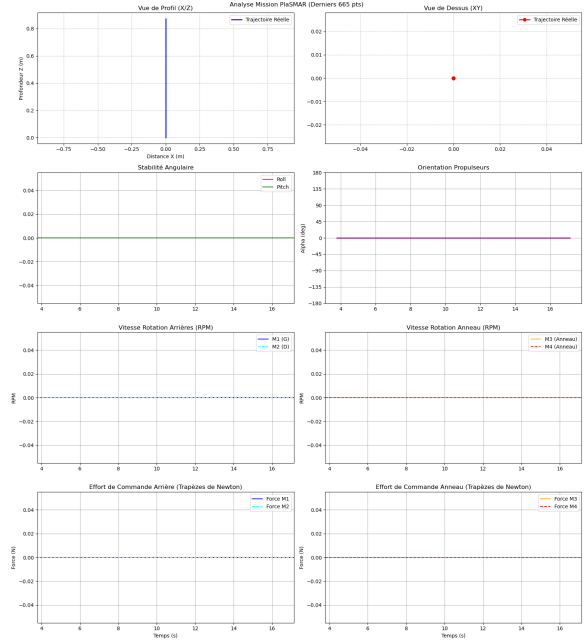


FIGURE 21 – Monitoring : Stabilité angulaire parfaite

##### Cas 2

Dans ce deuxième test, le robot est initialisé avec une gîte importante (Roll = 45°, Pitch = 45°). On observe l'oscillation amortie et le retour à l'équilibre, validant le positionnement du centre de gravité.

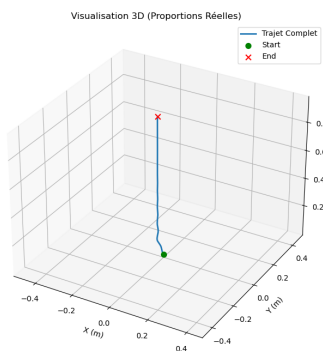


FIGURE 22 – Trajectoire de redressement 3D

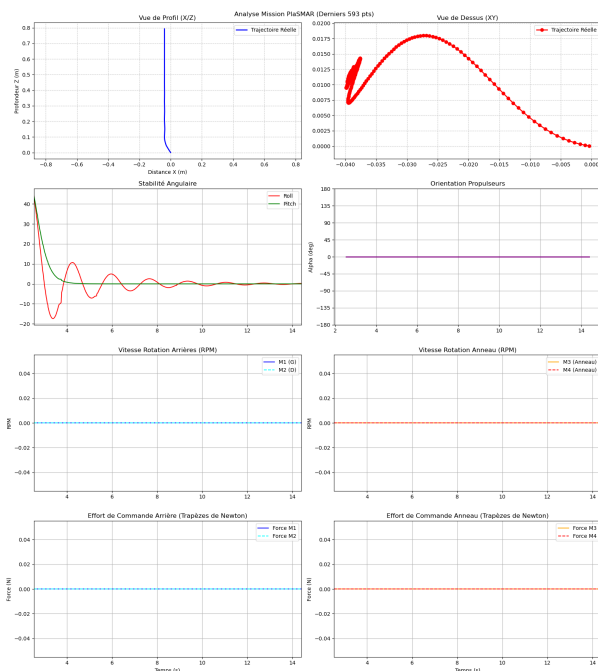


FIGURE 23 – Monitoring : Amortissement des oscillations

### A.2.2 Visualisations 3D des Mouvements Élémentaires

Ces vues complètent les résultats présentés dans la section 5.3. Elles permettent d'observer les trajectoires réelles du robot

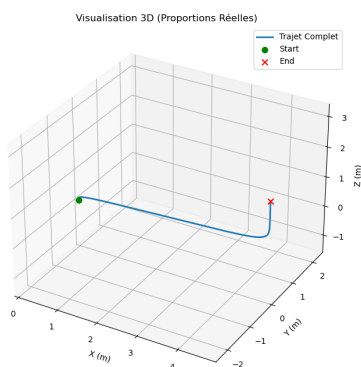


FIGURE 24 – Trajectoire 3D - Test Avance

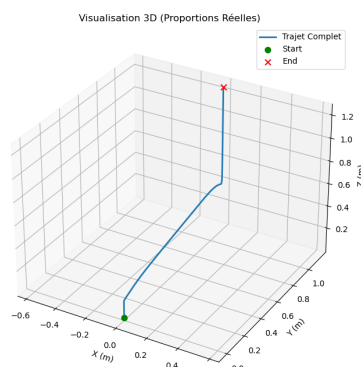


FIGURE 25 – Trajectoire 3D - Test Latéral

### A.2.3 Séquence de mouvement

Résultats détaillés de l'enchaînement de mouvements du robot : Avance → Latéral → Plongée → Avance → Remontée.

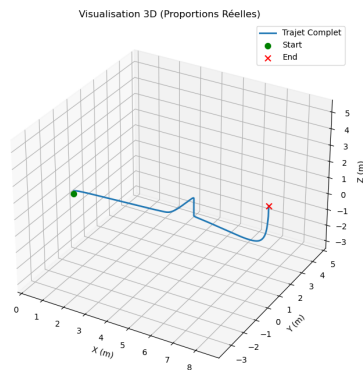


FIGURE 26 – Trajectoire globale de la mission en 3D

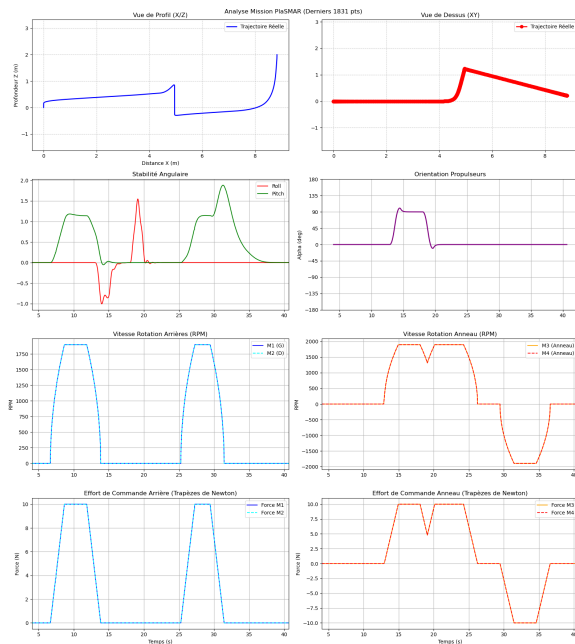


FIGURE 27 – Monitoring : Séquence

### A.3 Commandes aimants

Nous avons contacté divers fournisseurs pour des aimants sur mesure néodyme-fer-bore N42 ou N45 avec des dimensions précises (voir figure ??). Voici les différentes options disponibles à ce jour :

- **Osenc** : possible, prix inconnu  
<https://osenc.com/fr/>
- **Magnet Store** : possible, 140 €  
[https://magnetstore.co.uk/?srsltid=AfmB0orFMqB1Sbt5hL\\_Tl0oszPKFeRdcxWDb1fTtqw60j](https://magnetstore.co.uk/?srsltid=AfmB0orFMqB1Sbt5hL_Tl0oszPKFeRdcxWDb1fTtqw60j)
- **First Magnetic France** : à voir, prix inconnue  
<https://www.firstmagneticfrance.com/>
- **Newland Magnetics** : possible, 570 €  
<http://www.newlandmagnetics.eu/>

